CampusGuard AI

Suspended Student Detection & Alert System Using Face Recognition

Design & Development Brief — prepared for Google Antigravity

Project Type	Final-Year Engineering Project — Full System
Reference Basis	"Detection of Missing Persons using Face Recognition and AI in Video Surveillance" (adapted)
Core Technique	MTCNN face detection + FaceNet/ArcFace embeddings + cosine/FAISS matching
Stack	Django + Django REST Framework + face_recognition/OpenCV + SQLite + Bootstrap
Current Status	Design 100% complete · Implementation ~10% (models scaffolded, rest pending)
Document Purpose	Give Antigravity full context to scaffold, generate, and complete the codebase

How to Use This Document (Read Me First)

This brief is written for **Google Antigravity** (an agentic IDE/coding assistant) to design and develop **CampusGuard AI** end-to-end. All planning and architecture decisions below were finalized in an earlier design conversation and should be treated as **settled requirements**, not open questions. Antigravity's job is to take this specification and produce a working Django codebase: project scaffolding, models, registration flow, the live-detection pipeline, the alert engine, and the admin dashboard.

Where a decision genuinely needs to be made while coding (e.g. a specific library version, a folder-naming convention), Antigravity should choose a sensible default consistent with the stack below and proceed, rather than stalling on it.

What to build, in order

- **Step 1 — Project scaffolding:** Django project + app structure, settings, URLs, static/media config.
- **Step 2 — Data layer:** the three core models (already specified in Section 5) and admin registration.
- **Step 3 — Registration flow:** photo upload → preprocessing → embedding generation → save to DB.
- **Step 4 — Detection pipeline:** the `run_surveillance` management command described in Section 6.
- **Step 5 — Alert engine:** Gmail SMTP email function with photo attachment and case details.
- **Step 6 — Dashboard:** Bootstrap templates for the two review queues plus case management views.
- **Step 7 — Report & slides:** optional — an IEEE-style project report and a review presentation.

1. Problem Statement & Objectives

Educational campuses that suspend students for disciplinary reasons currently have no automated way to detect if a suspended student re-enters campus premises during their suspension period. Manual gate checks depend entirely on security staff recognizing faces from memory or paper notices, which is unreliable at scale and easy to circumvent. CampusGuard AI closes this gap by continuously matching faces captured at campus entry points against a maintained database of currently-suspended students, and alerting the relevant authority the moment a high-confidence match occurs.

1.1 Objectives

- Detect and recognize faces from a live CCTV/webcam feed at campus gates in real time.
- Match detected faces against a maintained database of suspended students using face-embedding similarity.
- Apply a tiered confidence-threshold system to minimize false positives before any human sees an alert.
- Route matches through a human verification step — no alert is sent without admin confirmation.
- Notify the security office, HOD, or warden automatically once a match is verified.
- Maintain a complete, timestamped audit log of every detection for accountability and reporting.

1.2 Scope Boundaries (what this project is — and isn't)

In Scope	Out of Scope (this iteration)
Single or multi-camera coverage at defined gate/entry points	Campus-wide CCTV network integration (hallways, classrooms)
Face detection + recognition against a known suspended-student list	Open-set person re-identification / unknown-person tracking
Web-based admin dashboard for verification and case management	Mobile app for security staff
Email/SMS alerting to a fixed set of authority roles	Integration with campus ID-card / biometric attendance systems

2. Mapping from the Reference Paper

The design is adapted from the reference paper *"Detection of Missing Persons using Face Recognition and AI in Video Surveillance."* The core computer-vision pipeline (face detection → embedding → similarity matching → thresholded alerting) transfers directly; only the domain context, data source, and notification target change.

Missing-Person Paper	CampusGuard AI
Public upload of sighting photos	Continuous CCTV/webcam feed at campus gates
Missing-persons database	Suspended-students database (photo + name + suspension details)
MTCNN face detection	Same — MTCNN (or dlib HOG as a lighter fallback)
FaceNet / ArcFace embeddings	Same
Cosine similarity / FAISS matching	Same — match live face against suspended-student embeddings
Low/high-confidence threshold split	Same logic (e.g. below 70% ignore, above 85% auto-alert)
Admin portal verifies match	Security/faculty admin verifies detection before alert is sent
Email alert to family	Alert to security office / HOD / warden (email, extensible to SMS)
Django backend + dashboard	Same — Django + DRF APIs + Bootstrap dashboard

Core pipeline (one line): Camera feed → Face Detection (MTCNN) → Embedding (FaceNet/ArcFace) → Match against suspended-student DB (FAISS/cosine) → Confidence threshold → Admin alert → Verify → Notify security → Log entry with location/timestamp.

3. System Architecture

The system is organized into three functional subsystems that map to distinct concerns: capturing and preprocessing input, recognizing and matching faces, and managing the human-verified alert workflow. This separation lets each subsystem be developed, tested, and swapped independently (e.g. replacing MTCNN with a lighter detector, or FAISS with a plain cosine loop for a small student database).

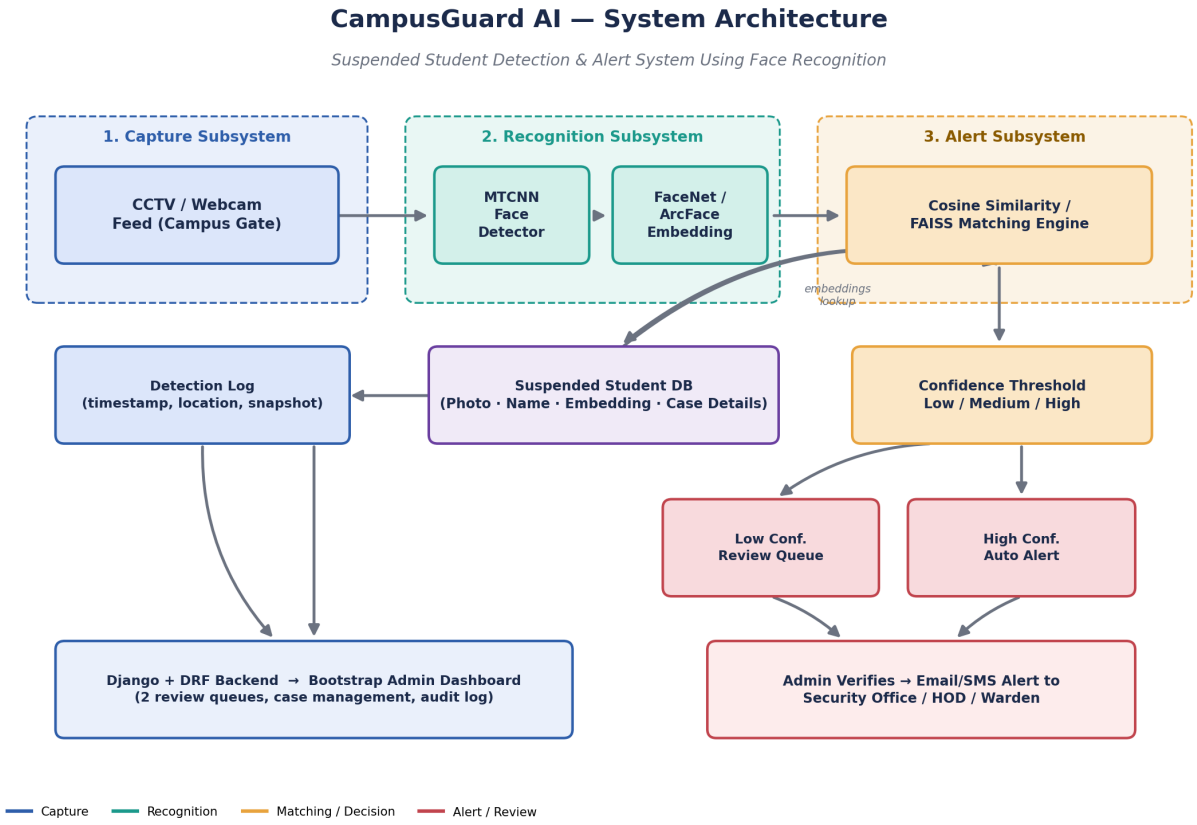

Figure 1. End-to-end system architecture across the three subsystems.

3.1 Subsystem Responsibilities

Subsystem	Responsibility	Key Technologies
Capture Subsystem	Acquire and sample frames from gate cameras; hand frames to the recognition pipeline.	OpenCV, RTSP/webcam input
Recognition Subsystem	Detect faces, generate embeddings, and compute similarity against the suspended-student database.	MTCNN, FaceNet/ArcFace, FAISS/cosine
Alert Subsystem	Apply confidence thresholds, route to review queues, and send verified alerts to authorities.	Django, DRF, Gmail SMTP, Bootstrap

4. Detection Pipeline (Runtime Logic)

This is the exact per-frame decision logic that the `run_surveillance` management command must implement. It is the single most important piece of runtime behavior in the system — Antigravity should treat this flowchart as the authoritative spec for that command.

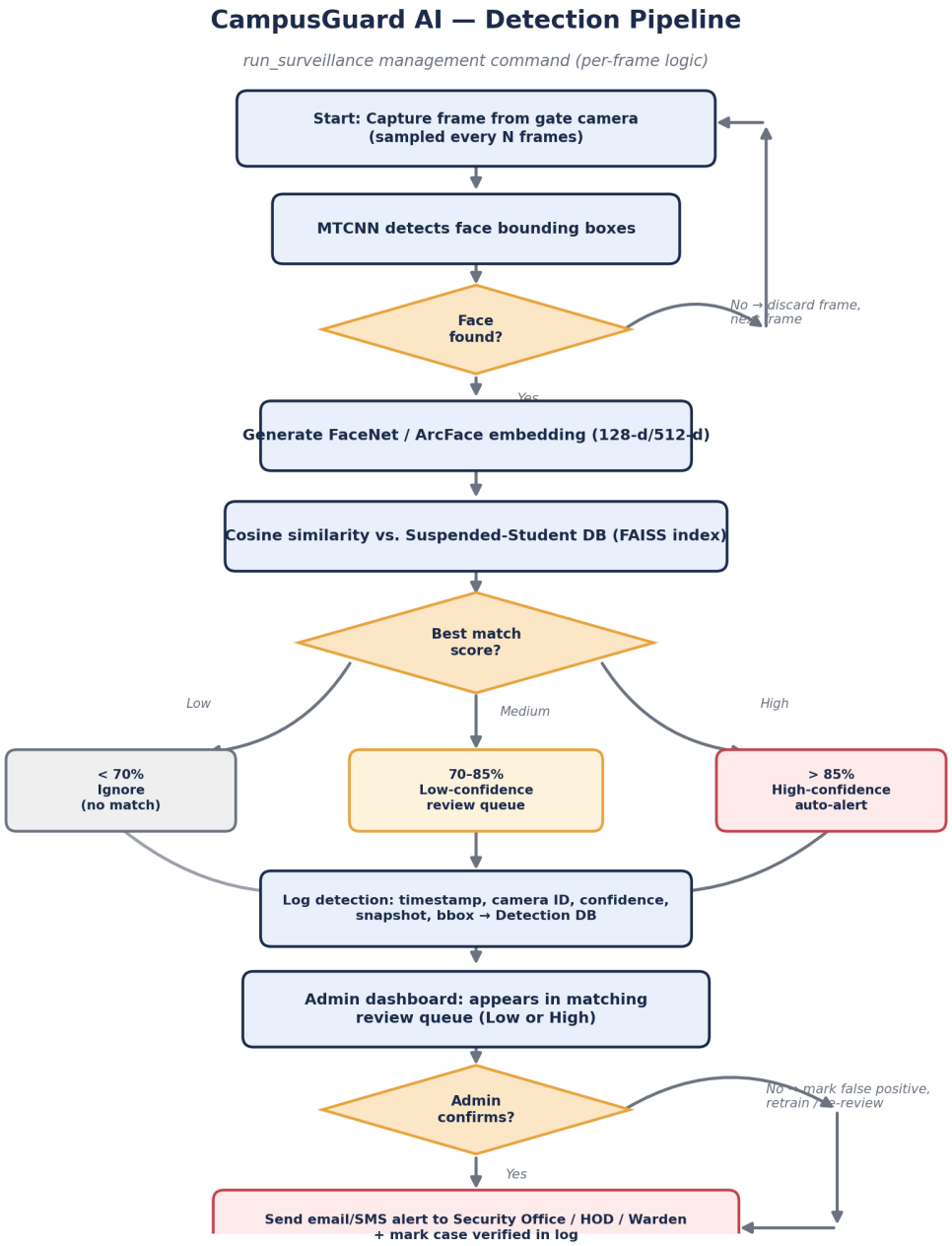

Figure 2. Per-frame detection and alert-routing pipeline.

4.1 Confidence Thresholds

Similarity Score	Classification	System Behavior
< 70%	No match	Discard silently. Not logged as a match (frame may still be logged for debugging).
70% – 85%	Low-confidence	Goes to the Low-Confidence Review Queue. Admin must actively confirm before any alert.
> 85%	High-confidence	Goes to the High-Confidence queue for fast admin sign-off; still requires one-click confirmation — never auto-sent.

Design principle: no notification ever leaves the system without a human admin explicitly confirming the match. This is a deliberate safeguard against false accusations reaching security staff, parents, or disciplinary committees on the basis of an automated match alone.

4.2 Frame Sampling & Performance Notes

- Process every N-th frame (e.g. every 5th–10th frame at typical webcam FPS) rather than every frame, to keep the pipeline real-time on modest hardware.
- Cache/index suspended-student embeddings in memory (and rebuild the FAISS index) on service start and whenever the roster changes, not on every frame.
- Deduplicate repeat detections of the same person within a short time window (e.g. 2–5 minutes) so one visit doesn't generate dozens of queue entries.

5. Data Model

Three Django models form the persistence layer. `SuspendedStudent` is the roster of people to watch for; `FaceEmbedding` stores one or more reference vectors per student (multiple angles improve match robustness); `DetectionLog` is the append-only audit trail of every detection event and its resolution.

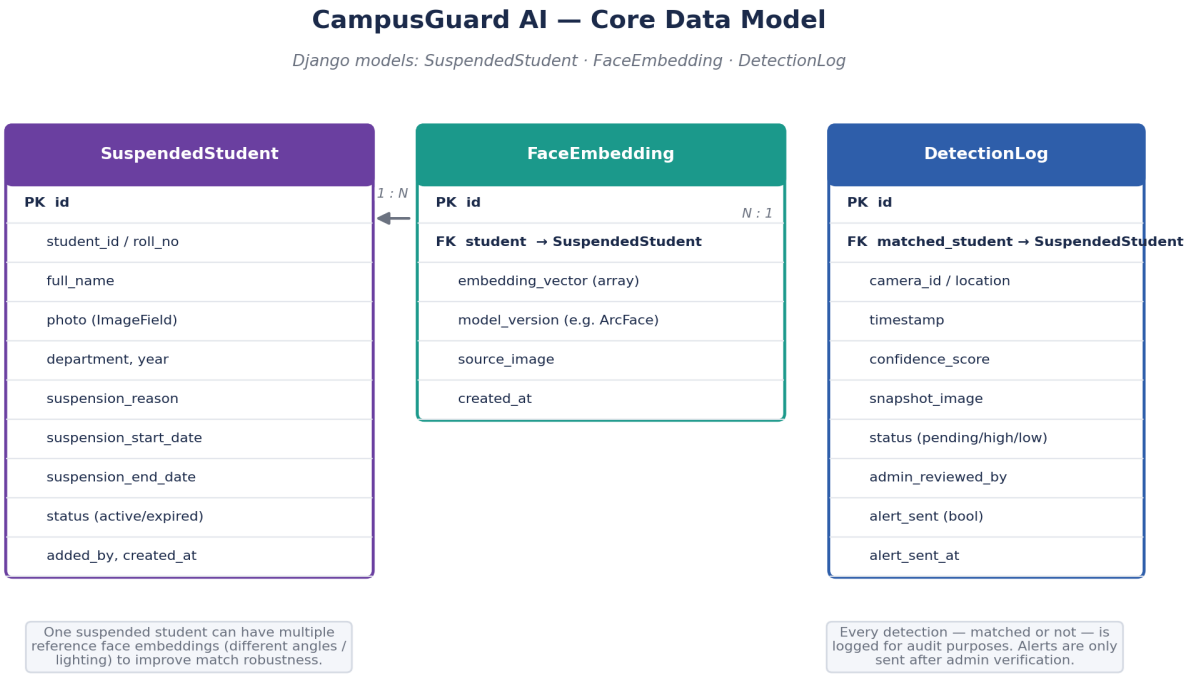

Figure 3. Core data model and relationships.

5.1 Reference Model Definitions

These reflect the models.py already scaffolded in the earlier design session. Antigravity should use these as the ground truth for field names when generating migrations, serializers, and forms.

```
SuspendedStudent
  id                    AutoField (PK)
  student_id            CharField       # roll number / college ID
  full_name             CharField
  photo                 ImageField
  department            CharField
  year                  CharField
  suspension_reason     TextField
  suspension_start_date DateField
  suspension_end_date   DateField
  status                CharField       # active / expired
  added_by              ForeignKey(User)
  created_at            DateTimeField(auto_now_add=True)

FaceEmbedding
  id               AutoField (PK)
  student          ForeignKey(SuspendedStudent, related_name="embeddings")
  embedding_vector JSONField / BinaryField  # serialized float vector
  model_version    CharField                # e.g. "arcface-r100"
  source_image     ImageField
  created_at       DateTimeField(auto_now_add=True)
```

```
DetectionLog
  id                AutoField (PK)
  matched_student   ForeignKey(SuspendedStudent, null=True, related_name="detections")
  camera_id         CharField
  location          CharField
  timestamp         DateTimeField(auto_now_add=True)
  confidence_score  FloatField
  snapshot_image    ImageField
  status            CharField  # pending/low_confidence/high_confidence/verified/rejected
  admin_reviewed_by ForeignKey(User, null=True)
  alert_sent        BooleanField(default=False)
  alert_sent_at     DateTimeField(null=True)
```

6. Module Breakdown

The system decomposes into 10 functional modules across 4 subsystems. This is the build checklist — each module below should become one or more Django apps, management commands, or template sets.

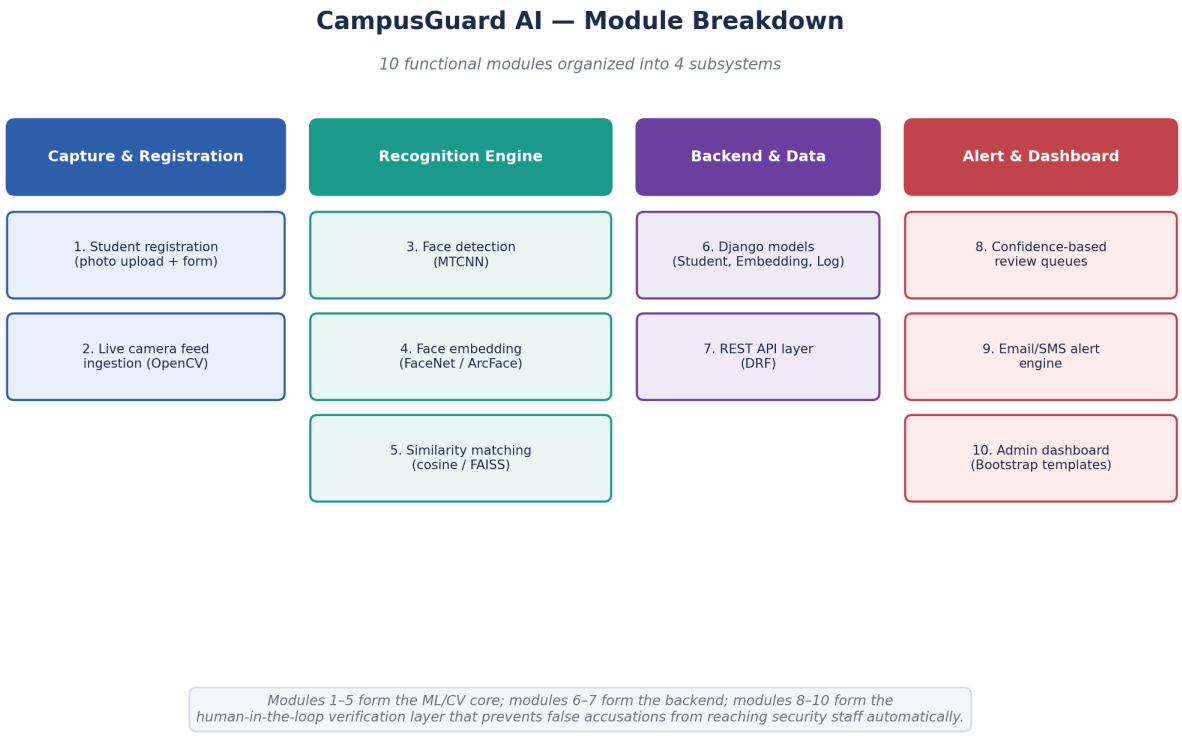

Figure 4. The 10 modules grouped by subsystem.

7. Technology Stack

Layer	Choice	Why
Backend framework	Django	Batteries-included ORM, admin panel, and auth — fast for a final-year timeline.
API layer	Django REST Framework (DRF)	Clean serialization for dashboard AJAX calls and any future mobile client.
Face detection	MTCNN (or dlib HOG fallback)	Matches the reference paper; MTCNN is accurate, dlib HOG is lighter on CPU-only machines.
Face embedding	FaceNet / ArcFace (via face_recognition or insightface)	Pretrained, no training data required for this scope.
Similarity search	Cosine similarity (small DB) / FAISS (larger DB)	Cosine is enough for a few hundred students; FAISS scales if needed.
Video capture	OpenCV	De facto standard for frame capture from webcam/RTSP/CCTV streams.
Database	SQLite (dev) → PostgreSQL (optional prod)	SQLite is zero-config for a college project demo; Postgres is a drop-in upgrade.
Frontend	Bootstrap + Django templates	No separate frontend build step needed; fast to style a functional dashboard.
Alerts	Gmail SMTP (django.core.mail)	Simplest reliable channel for a demo; SMS can be added later via a gateway API.

8. Implementation Status & Roadmap

Planning and design are complete. The table below is the authoritative build checklist for Antigravity — treat unchecked items as the actual task list, in roughly the order they should be built.

#	Deliverable	Status	Notes for Antigravity
—	Project title, abstract, architecture	**Done**	Use as-is; see Sections 1–3.
—	models.py (SuspendedStudent, FaceEmbedding, DetectionLog)	**Done**	Field spec in Section 5.1 — generate the actual file + migrations.
1	Django project scaffolding	**Pending**	Create project + apps (e.g. core, students, surveillance, alerts, dashboard).
2	Student registration flow	**Pending**	Upload form → face crop/align → embedding → save FaceEmbedding row.
3	run_surveillance management command	**Pending**	Implement Figure 2's logic exactly; make camera source configurable.
4	Email alert function	**Pending**	Gmail SMTP; template includes student photo, name, confidence, timestamp, location.
5	Admin dashboard views/templates	**Pending**	Two queues (low/high confidence) + case detail + resolve/reject actions.
6	REST API endpoints (DRF)	**Pending**	Expose queues and case actions for any AJAX/future mobile use.
7	Project report (IEEE-style)	**Pending**	Mirror the reference paper's structure; content sources are Sections 1–8 of this brief.
8	Review presentation (PPT)	**Pending**	One slide per major section; reuse Figures 1–4 directly.

9. Working Instructions for Antigravity

This section translates the specification above into concrete build instructions. Follow it directly when scaffolding and generating code.

9.1 Suggested Project Layout

```
campusguard/
|-- manage.py
|-- campusguard/            # project settings package
|   |-- settings.py
|   |-- urls.py
|   `-- wsgi.py / asgi.py
|-- students/               # SuspendedStudent + registration flow
|   |-- models.py
|   |-- forms.py
|   |-- views.py
|   `-- embeddings.py       # face preprocessing + embedding generation
|-- surveillance/           # detection pipeline
|   |-- management/commands/run_surveillance.py
|   |-- matcher.py          # cosine / FAISS matching logic
|   `-- models.py           # DetectionLog (or shared app)
|-- alerts/                 # email/SMS notification logic
|   `-- mailer.py
|-- dashboard/              # admin-facing views & templates
|   |-- views.py
|   `-- templates/dashboard/
|-- static/ & media/
`-- requirements.txt
```

9.2 Build Order & Acceptance Checks

- **Scaffold first:** get `python manage.py runserver` working with the three models registered in Django admin before writing any CV code.
- **Registration flow next:** confirm a photo upload produces a stored embedding you can print/inspect — this de-risks the CV dependencies early.
- **Pipeline in isolation:** build and test `run_surveillance` against a local webcam or a sample video file before wiring it to real camera hardware.
- **Alerts behind a flag:** keep email sending behind a settings flag / console-backend during development so testing the pipeline doesn't spam real inboxes.
- **Dashboard last:** once DetectionLog rows are being created correctly, the dashboard is mostly templating — build it against real logged data.

9.3 Non-Negotiable Constraints

- No automated alert may be sent without an explicit admin confirmation action (see Section 4.1).
- Every detection event — matched or not — should be recoverable from DetectionLog for audit purposes.
- Confidence thresholds (70% / 85%) must be configurable (settings.py or DB-backed config), not hardcoded magic numbers scattered through the codebase.
- Face images and embeddings are sensitive data — keep media files out of version control and out of any public-facing URL without auth.

This document is intended to be the single source of truth Antigravity needs to design and develop CampusGuard AI without further clarification. Where any ambiguity remains, prefer the simplest choice consistent with the stack in Section 7 and the constraints in Section 9.3.